

Abstimmung

28.05.2025



Diskussion der letzten Wochen



Implementierung in main() und
Beschreibungsdatei.



Modul-Klasse.



Kopplung und algebraische Schleifen.



Pläne bis Anfang Juni.



Pläne bis Ende Juni.

main() & description file

Nur eine Modul Klasse!

low_pass_filter.py

```
from decorators import differential

MODULE_INPUTS = ["Uin"]
MODULE_OUTPUTS = ["Uc"]

PARAMS = {
    "R": {"default": 1000, "unit": "Ohm"},
    "C": {"default": 1e-6, "unit": "F"}
}

INITIAL_VALUES = {
    "Uc": (0.0, "V")
}

@differential(outputs=["Uc"])
def dUc(t, Uin, Uc, R, C):
    return (Uin - Uc) / (R * C)
```

main.py

```
from framework import Module, Simulation

# Voltage Source (sine wave)
source = Module(
    name="Source",
    path="voltage_source.py", # description file
    parameters={"A": (1.0, "V"), "f": (5.0, "Hz")}
)

# Low-pass RC filter
lpf = Module(
    name="LPF",
    path="low_pass_filter.py",
    parameters={"R": (1000, "Ohm"), "C": (1e-6, "F")},
    initial_values={"Uc": (0.0, "V")} # optional; could be omitted
)

# RLC circuit in parallel
rlc = Module(
    name="RLC",
    path="rlc_parallel.py",
    parameters={"R": (100, "Ohm"), "L": (0.01, "H"), "C": (1e-6, "F")},
    initial_values={"IL": (0.0, "A")} # only override IL; Uc can come from upstream
)

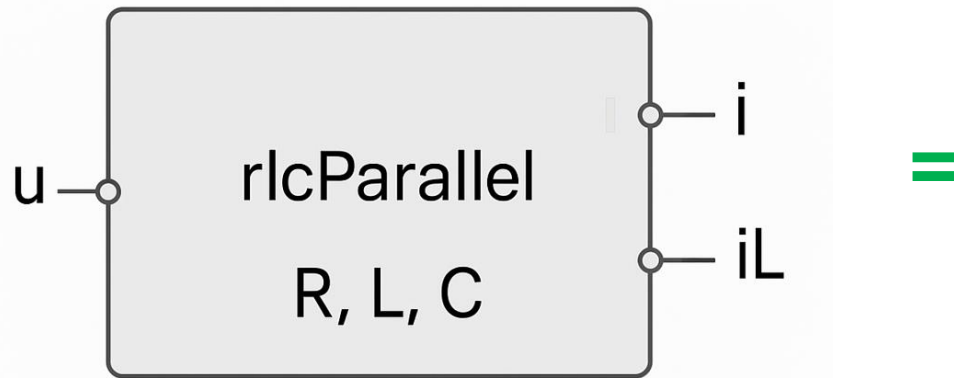
# Set up simulation
sim = Simulation(modules=[source, lpf, rlc])

# Connect modules
sim.connect(src="Source", output="Uin", dst="LPF", input="Uin")
sim.connect(src="LPF", output="Uc", dst="RLC", input="U")

# Run simulation and plot
sim.run()
sim.plot()
```

rlc_parallel_module.py

Funktionen im Modul werden mit
@Dekoratoren markiert!



PhysicalValue Klasse
im Hintergrund verwendet!

```
from decorators import algebraic, differential

MODULE_INPUTS = ["U"]
MODULE_OUTPUTS = ["I", "IL"]

PARAMS = {
    "R": {"default": 100, "unit": "Ohm"},
    "L": {"default": 0.01, "unit": "H"},
    "C": {"default": 1e-6, "unit": "F"}
}

INITIAL_VALUES = {
    "IL": (0.0, "A")
}

@algebraic(outputs=["I"])
def compute_total_current(t, U, IL, dU, R, C):
    return U / R + C * dU + IL

@differential(outputs=["IL"])
def dIL(t, U, L):
    return U / L
```

Was macht die Modulklassse?

1. Vorhersage (Prediction): `solve_ivp` gelöst $\rightarrow$ Zustände $y[k+1]$.
 2. Algebraische Gleichungen: `scipy.root()` $\rightarrow$ $z[k+1]$.
 3. Common-Funktionen: Unabhängig gelöst– sie verarbeiten Daten und sind nicht gekoppelt.
 4. Jedes Modul speichert nur den aktuellen Wert, die gesamte Zeitschritthistorie wird in der Simulation geführt:
 - $y[k+1]$ und $z[k+1]$ bleiben im Modul; $y[k]$ und $z[k]$ sind weitergeschickt.
 - Outputs sind im Zeitschritt weitergeschickt.
 - Nur deklarierte Outputs sind nach außen sichtbar.
-

Kopplung von Modulen

Kopplung entsteht, wenn zwei oder mehr Module wechselseitig innerhalb eines Zeitschritts voneinander abhängen → algebraische Schleife.

Gekoppelte Module werden intern mithilfe der Bibliothek **NetworkX** erkannt und automatisch in einer **CoupledBlock**-Klasse zusammengefasst.

CoupledBlock wird anschließend wie ein normales Modul in der **Ausführungsreihenfolge** behandelt.

Am Ende werden alle berechneten Outputs den jeweiligen Modulen zurückgegeben.

Lösestrategien im CoupledBlock

Fall	Enthaltene Gleichungen	Vorgehen im CoupledBlock
Nur algebraische Gleichungen	<code>algebraic(...)</code>	Gleichungssystem mit <code>scipy.root()</code> lösen
Nur differentielle Gleichungen	<code>differential(...)</code>	Integration mit <code>solve_ivp()</code>
Algebraisch + Differentiell	beide Typen vorhanden	Schritt 1: <code>solve_ivp()</code> → Schritt 2: <code>root()</code>
Nur Common-Funktionen	<code>common(...)</code>	Direkte Auswertung, kein Solver erforderlich

Bis Anfang Juni

- Ein **Framework**, das Module mit **differentiellen und algebraischen Gleichungen** lösen kann – auch im **gekoppelten Fall**.
- Module könnten **unterschiedliche Zeitschritte** verwenden.
- Ein **Simulationsmanager** würde die Ausführung koordinieren, Verbindungen auflösen und die Ergebnisse plotten.
- Interne Variablen.
- Ein **Benutzermenü**.
- Beispiel an einer elektrischen Schaltung.

Bis Ende Juni

- Eine **Visualisierung**.
- Planen und erläutern Sie, wie der Framework andere Tools integrieren würde (**OpenSim und FEM**) und wenn es die Zeit erlaubt, ein Beispiel implementieren
 - *oder Simulink oder OpenModelica?*
- Wenn es die Zeit erlaubt, soll **eine einfachere Beinprothesensimulation** auf der Basis eines Doppelpendels realisiert werden.

OpenModelica vs Simulink

1

OpenModelica: Klassen,
Hierarchie, Hintergrund.

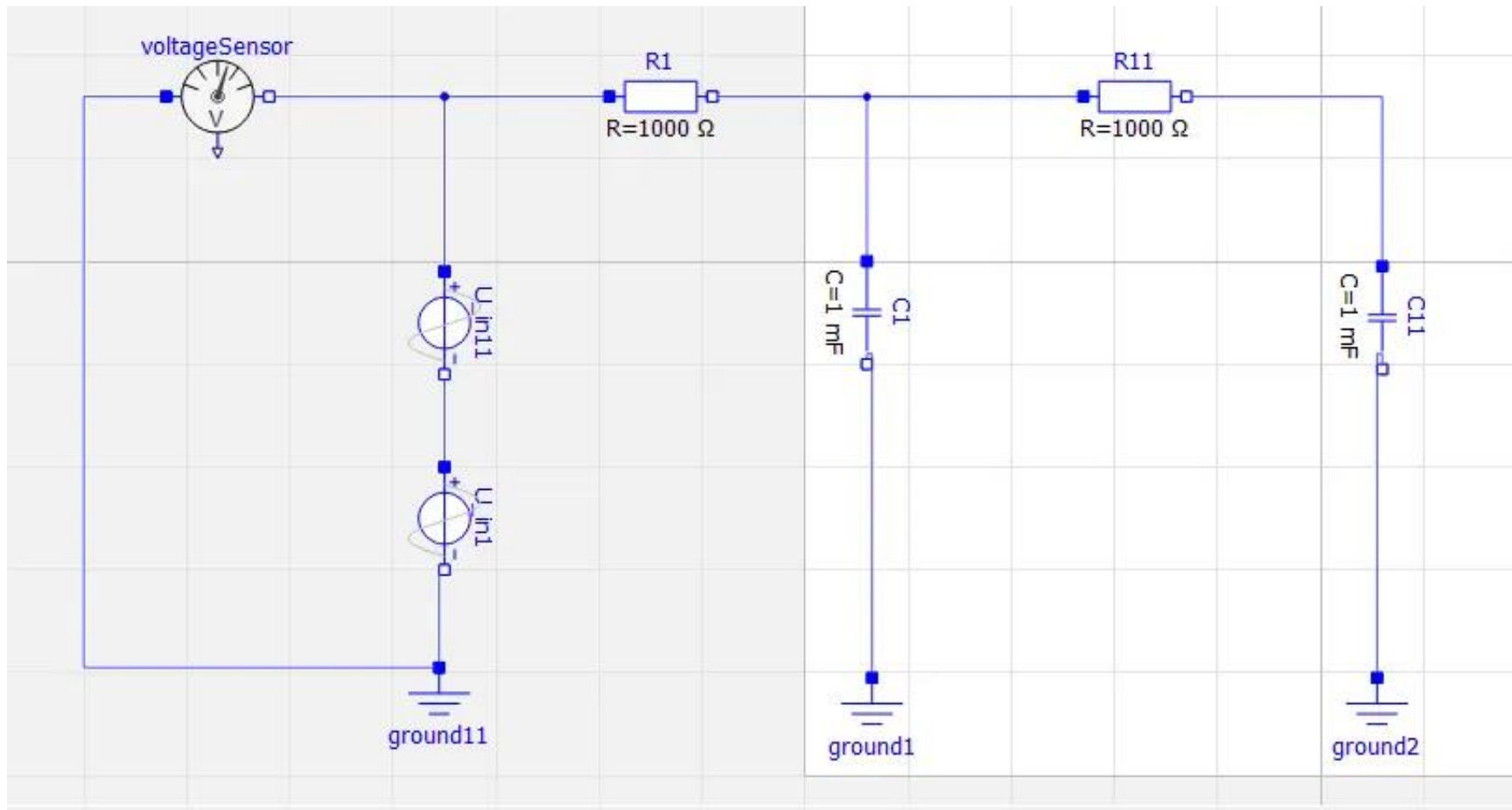
2

Simulink: Signalfluss,
Hierarchie.

3

Signal-based modeling vs
equation-based modeling

OpenModelica – Tiefpass Filter 2. Ordnung



Schnittstellen in Modelica

Schnittstellen werden durch die **Verbindung von Objekten der Connector-Klasse** gebildet.

Connectors stellen **immer dieselbe Art von Beziehungen** her (im Fall der elektrischen Eigenschaften handelt es sich um U und I).

Vorteil: Einfaches Herstellen von Verbindungen.

Nachteil: Inflexibilität. Man kann sich nicht aussuchen, welche Teile des Verbinders man verbindet - **man bekommt alles oder nichts.**

```
modelica

connector Pin
  Real v;          // non-flow → equalized across connection
  flow Real i;     // flow → summed to zero across connection
end Pin;
```

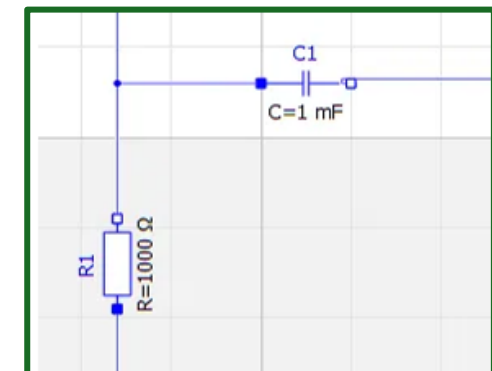
Then if you do:

```
modelica

connect(R1.n, C1.p); // Both are Pins
```

Modelica enforces:

- $R1.n.v = C1.p.v$
- $R1.n.i + C1.p.i = 0$



Modelica - Model Klasse

Alle elektrischen Elemente werden durch **ein gemeinsames Modell erweitert**, das ihr Verhalten bestimmt.

```
partial model OnePort
  Pin p, n;
  Voltage v;
  Current i;
equation
  v = p.v - n.v;
  0 = p.i + n.i;
  i = p.i;
end OnePort;
```

Einzelne Elemente definieren nur Gleichungen, die für sie spezifisch sind.

```
model Capacitor "Ideal electrical capacitor"
  extends OnePort;
  parameter Real C(unit="F") "Capacitance";
equation
  C*der(v) = i;
end Capacitor;
```

der() bezieht sich auf die Ableitung.

Modelica - Real Class: behandelt die Variablen

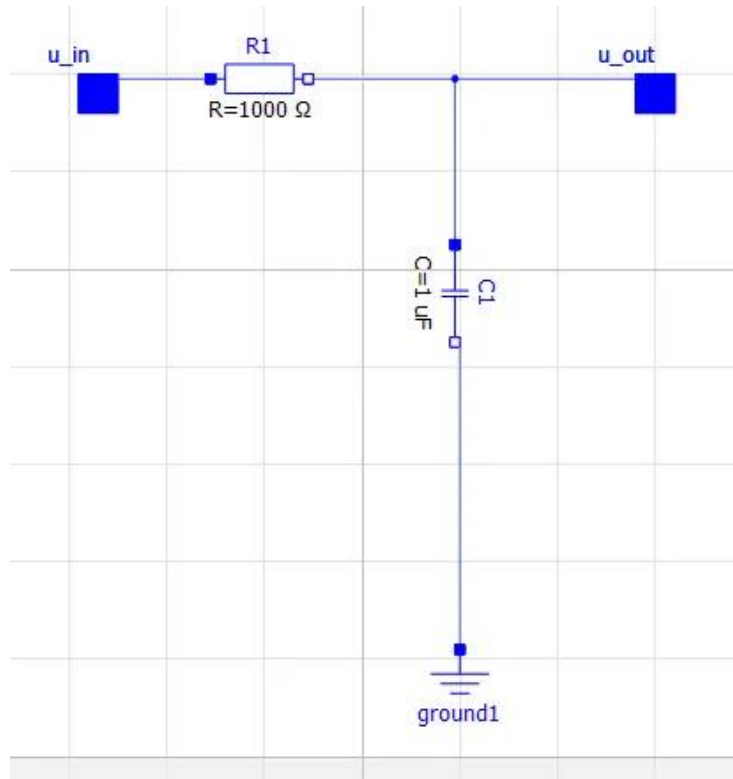
A real variable has a set of attributes such as unit of measure, initial value, minimum and maximum value.

```
model Resistor "Ideal electrical resistor"
  extends OnePort;
  parameter Real R(unit="Ohm") "Resistance";
equation
  R*i = v;
end Resistor;
```

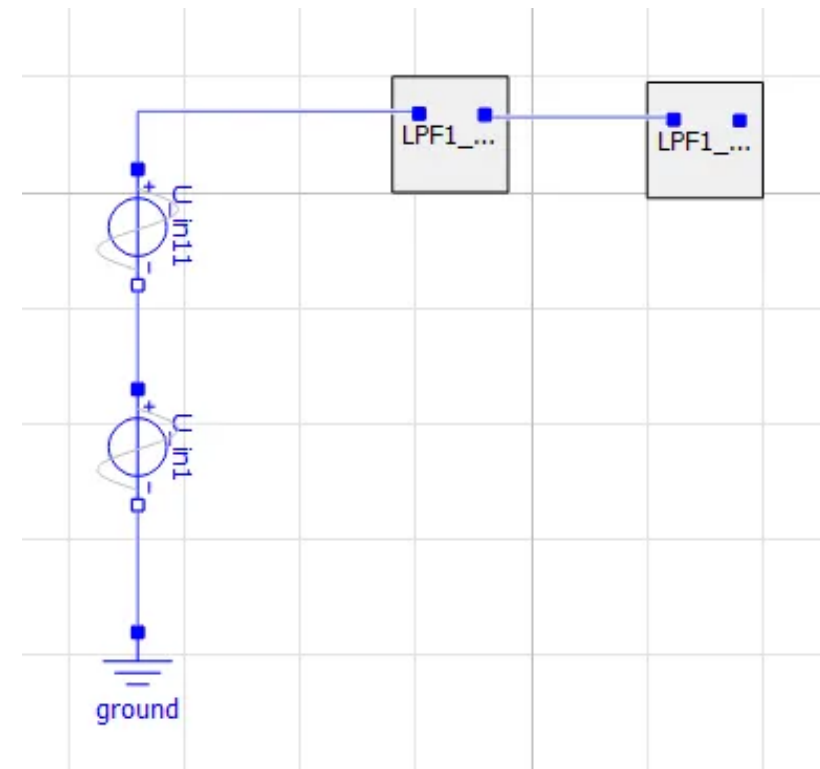
```
model VsourceAC "Sin-wave voltage source"
  extends OnePort;
  parameter Voltage VA = 220 "Amplitude";
  parameter Real f(unit="Hz") = 50 "Frequency";
  constant Real PI=3.141592653589793;
equation
  v = VA*sin(2*PI*f*time);
end VsourceAC;
```

Modulare Hierarchie in Modelica.

Tiefpass Filter 1. Ordnung



Tiefpass Filter 2. Ordnung



Die Pins bestimmen IOs und werden verwendet, um einen Block mit dem Rest des Systems zu verbinden.

Modelica-Syntax und ihre Funktionsweise

- Elektrische Elemente (wie R, C) sind **vordefiniert**. Sie können verwendet werden, um **größere Modelle zu erstellen**.
- Jedes Element enthält eine Reihe von Gleichungen, **die Äquivalenz an seinen Polen definieren** (für Spannung, Strom usw.).
- **Verbindungen stellen Gleichheit für bestimmte Werte an den Polen her.**
- Es sind nur bestimmte Arten von Verbindungen zulässig (z. B. keine Verbindung zweier negativer Pole).
- **Modelica erstellt dann aus diesen Gleichungen ein DAE-System und übersetzt es in C-Code, um eine schnellere Simulation zu ermöglichen.**

```
model DCMotor
```

```
  Resistor R(R=100);
```

```
  Inductor L(L=100);
```

```
  VsourceDC DC(f=10);
```

```
  Ground G;
```

```
  ElectroMechanicalElement EM(k=10,J=10, b=2);
```

```
  Inertia load;
```

```
equation
```

```
  connect(DC.p,R.n);
```

```
  connect(R.p,L.n);
```

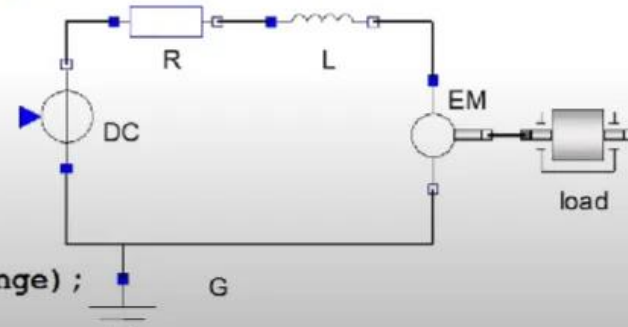
```
  connect(L.p, EM.n);
```

```
  connect(EM.p, DC.n);
```

```
  connect(DC.n,G.p);
```

```
  connect(EM.flange,load.flange);
```

```
end DCMotor
```



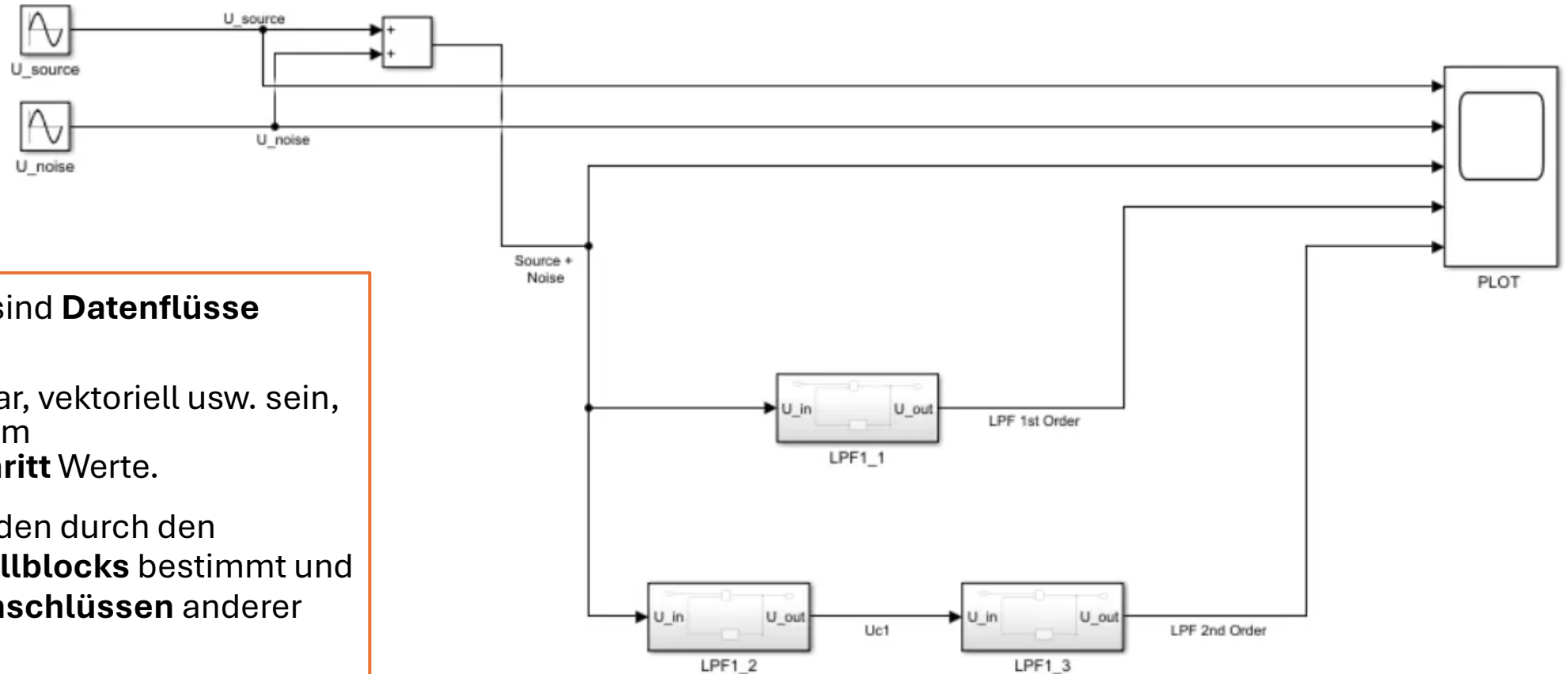
The following equations are automatically derived from the Modelica model:

$0 == DC.p.i + R.n.i$	$EM.u == EM.p.v - EM.n.v$	$R.u == R.p.v - R.n.v$
$DC.p.v == R.n.v$	$0 == EM.p.i + EM.n.i$	$0 == R.p.i + R.n.i$
	$EM.i == EM.p.i$	$R.i == R.p.i$
$0 == R.p.i + L.n.i$	$EM.u == EM.k * EM.\omega$	$R.u == R.R * R.i$
$R.p.v == L.n.v$	$EM.i == EM.M / EM.k$	
	$EM.J * EM.\omega == EM.M - EM.b * EM.\omega$	$L.u == L.p.v - L.n.v$
$0 == L.p.i + EM.n.i$		$0 == L.p.i + L.n.i$
$L.p.v == EM.n.v$	$DC.u == DC.p.v - DC.n.v$	$L.i == L.p.i$
	$0 == DC.p.i + DC.n.i$	$L.u == L.L * L.i'$
$0 == EM.p.i + DC.n.i$	$DC.i == DC.p.i$	
$EM.p.v == DC.n.v$	$DC.u == DC.Amp * \sin[2 \pi DC.f * t]$	
$0 == DC.n.i + G.p.i$		
$DC.n.v == G.p.v$		(load component not included)

Automatic transformation to ODE or DAE for simulation:

$$\frac{dx}{dt} == f[x, u, t] \quad g\left[\frac{dx}{dt}, x, u, t\right] == 0$$

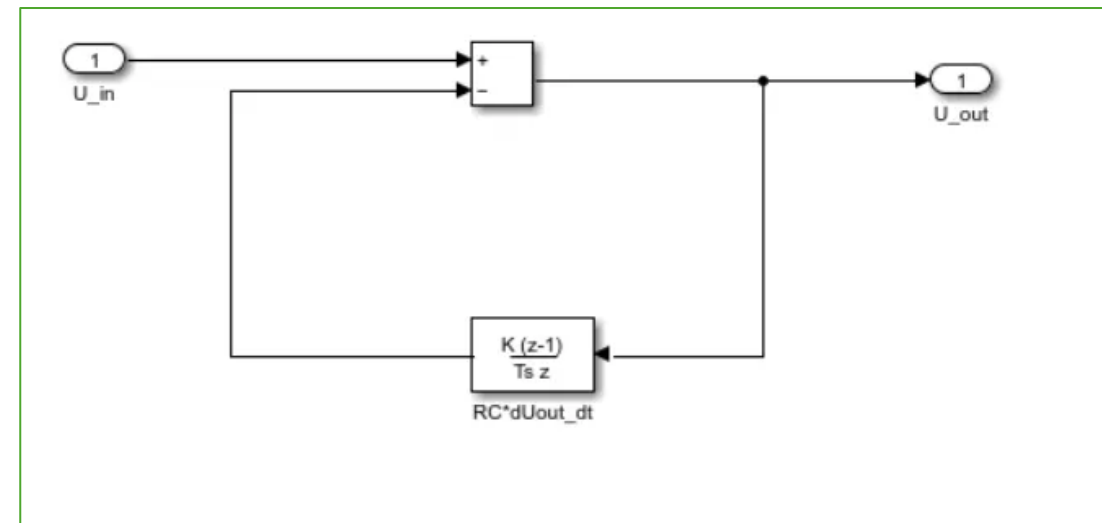
Simulink - Tiefpass Filter 2. Ordnung



- Signale in Simulink sind **Datenflüsse** zwischen Blöcken.
- Ein Signal kann skalar, vektoriell usw. sein, und es trägt bei jedem **Simulationszeitschritt** Werte.
- Die Signalwerte werden durch den **Ausgang eines Quellblocks** bestimmt und von den **Eingangsanschlüssen** anderer Blöcke verwendet.
- Signale können nur mit speziellen Blöcken **visualisiert** werden.

Simulink - Subblöcke

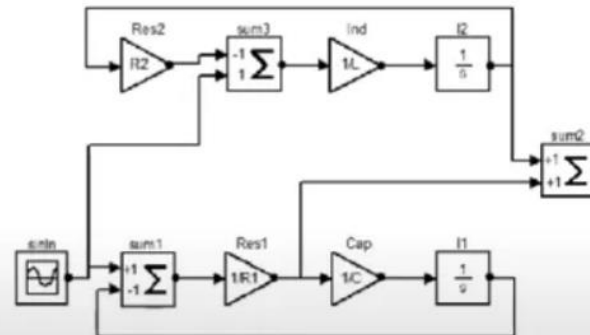
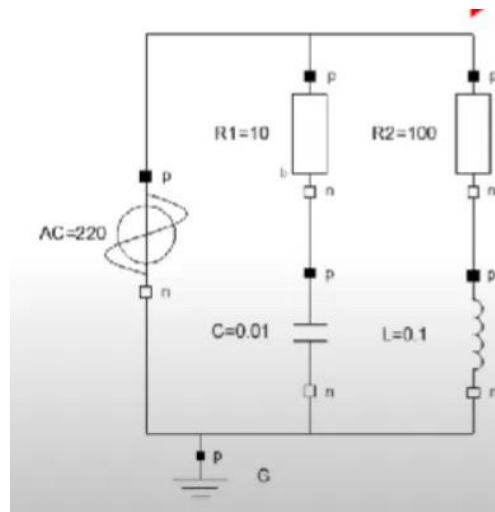
- Simulink ermöglicht es uns, einen Subblock mit **definierten Eingängen und Ausgängen zu erstellen**.
- Der Subblock TPF der 1. Ordnung wurde hier erstellt, wobei der Eingang die **Quelle + Rauschen** und der Ausgang die **Spannung am Kondensator** ist.
- Der Subblock stellt diese Formel dar:
$$dU_{out}/dt = (U_{in} - U_{out})/RC$$
- Ein TPF 2. Ordnung wurde durch einfaches **Verbinden zweier Kopien** des TPF1-Unterblocks erstellt.



Signal-based modeling vs equation-based modeling

- Im Fall von EBM haben wir **eher Gleichheit als Kausalität!**
- $U = R \cdot I$ und $I = U/R$ ist **das Gleiche** für EBM.
- Ideal für **physikalische** Systeme.
- **Vorteil:** intuitiv und einfach.
- **Nachteil:** Rigidität.

- SBM definiert Verbindungen klar. **Die Reihenfolge ist wichtig!**
- Z.B. mod1.output3 $\rightarrow$ mod2.input1 - **kausal**.
- Funktioniert gut mit **Kontrollsystemen!**
- **Vorteil:** Die Funktionsweise des Modells ist aus dem Diagramm ersichtlich.
- **Nachteil:** nicht intuitiv und es kann Zeit kosten, das System zu formulieren.



Unser Framework: Zwischen SBM und EBM

SBM-Elemente (Datenfluss)

Klare Deklaration von Inputs und Outputs pro Modul

Datenfluss erfolgt explizit über Verbindungen zwischen Modulen

Ausführung erfolgt zeitdiskret pro Zeitschritt

Pull-basiertes Datenmodell: Module fordern Daten von Parentmodule Modulen an

EBM-Elemente (CoupledBlock)

Gleichungen in gekoppelten Modulen werden gemeinsam gelöst $f(z) = 0$

Kein fester Signalfluss in CoupledBlocks

Danke!